

Reviewer Pack — Minimal Quadratic Residue Symbol and DPLL Search Tree Diamet...

Entry e2eb65a49cd5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 14:17:28 UTC

Minimal Quadratic Residue Symbol and DPLL Search Tree Diameter

Entry ID: e2eb65a49cd5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 14:17:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Number Theory **Field B** (complexity object): DPLL Search Trees

Statement:

For every Boolean satisfiability problem instance φ with n variables, the minimal quadratic residue symbol ($\text{qrs}(\varphi)$) of its associated polynomial is linearly correlated with its DPLL search tree diameter $d(\varphi)$, such that $\text{qrs}(\varphi) = \Theta(d(\varphi))$.

Rationale (proposer's reasoning):

Algebraic number theory provides a rich set of invariants for polynomials over finite fields, which can potentially capture structural information about the complexity of Boolean satisfiability problems. The quadratic residue symbol is a well-known invariant that could relate to the structure of DPLL search trees.

Taxonomy category: algebraic_number_theory_dpll (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 02e136a6f00e0785

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between $\text{qrs}(\varphi)$ and $d(\varphi)$ for each instance φ is ≥ 0.5 , calculated over 30 random seeds, and the p-value from a Pearson correlation test is ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Quadratic Residue Symbol algebraic number theory AND DPLL search tree diameter - DPLL search tree diameter in Boolean satisfiability problems related to quadratic residue symbol - algebraic number theory applications in DPLL search tree analysis for quadratic residue symbol

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_phi(n):
    phi = []
    for _ in range(n):
        clause = set()
        while len(clause) < 3:
            var = random.choice(range(1, n + 1))
            if var not in clause:
                clause.add(var)
        phi.append(list(clause))
    return phi

def polynomial(phi):
    p = [0] * (2 ** len(phi))
    for i in range(len(phi)):
        for assignment in range(2 ** len(phi[i])):
            product = 1
            for var in phi[i]:
                if (assignment >> (var - 1)) & 1:
                    product *= -1
            p[assignment] += product
    return p

def minimal_quadratic_residue_symbol(p):
    n = len(p)
    qrs = 0
    for i in range(n):
        if p[i] != 0:
            qrs = (qrs + p[i]) % 2
```

```

    return qrs

def dpll_search_tree(phi, assignment=0, path=[]):
    if len(path) == len(phi):
        return 1 if all(p >= 0 for p in polynomial(phi)) else 0

    var = phi[len(path)][0]
    count = 0
    count += dpll_search_tree(phi, assignment | (1 << (var - 1)), path + [True])
    count += dpll_search_tree(phi, assignment & ~(1 << (var - 1)), path + [False])
    return count

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        phi = generate_phi(n)
        p = polynomial(phi)
        qrs = minimal_quadratic_residue_symbol(p)
        d = dpll_search_tree(phi)
        results.append((qrs, d))

    if not results:
        return {
            "metric_name": "correlation",
            "metric_value": 0.0,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    n_total = len(results)
    qrs_values = [qrs for qrs, _ in results]
    d_values = [d for _, d in results]

    mean_qrs = sum(qrs_values) / n_total
    mean_d = sum(d_values) / n_total

    correlation = 0.0
    for qrs, d in results:
        correlation += (qrs - mean_qrs) * (d - mean_d)
    correlation /= (n_total * math.sqrt(sum((qrs - mean_qrs) ** 2 for qrs in qrs_values)) * math.sqrt(sum((d - mean_d) ** 2 for d in d_values)))

    return {
        "metric_name": "correlation",
        "metric_value": correlation,
        "instances_tested": n_total,
        "n_max": max(n_values),
        "conjecture_holds": abs(correlation) >= 0.5,
        "counterexample": ""
    }

if __name__ == "__main__":

```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]

results = []
for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

mean_correlation = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if abs(result["metric_value"]) >= 0.5) / len(results)

if all(abs(result["metric_value"]) >= 0.5 for result in results):
    print(f"RESULT: SUPPORTED mean={mean_correlation} std=0.0 support_fraction={support_fraction}")
elif any(abs(result["metric_value"]) < 0.5 for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if abs(result["metric_value"]) < 0.5)
    print(f"RESULT: FALSIFIED counterexample=\"correlation_too_low\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the correlation coefficient and p-value could not be calculated to verify the conjecture. | next: Run the test again with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	57569
2	preregistration	ollama_remote	glm4:latest	0	0	9795
3	novelty	ollama_remote	glm4:latest	0	0	9880
4	novelty	ollama_remote	glm4:latest	0	0	20533
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16023

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26663
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16255
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12968
9	judge	ollama_remote	glm4:latest	0	0	15503

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 185188 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/e2eb65a49cd5.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e2eb65a49cd5.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e2eb65a49cd5.tar.gz (if generated)